Пятнадцать месяцев финтеха агентами

что стало с качеством кода

Александр Черепанов
бэкенд-разработчик на Go, 12 лет опыта
с мая 2025 весь код в проде пишут агенты
автор открытого статического анализатора glint

Признание

У меня 12 лет опыта в Go.
И я больше не имею права называть себя программистом.

Мой код не пройдёт ту планку качества, которую я выставил для своих агентов.

Честнее так: они сами её выставили, я лишь ставил цели.



График вместо заявления



Красная пунктирная линия - уровень моего лучшего рукописного кода. Синяя кривая - код, который пишут агенты.

В апреле 2026 кривая ушла под красную линию и продолжает падать.

Сеттинг

- Закрытый финтех-продукт, веду один с мая 2025
- С февраля 2026 - второй проект, платёжный
- Код с первого дня пишут ИИ-агенты
- Модель делает главную работу, планку качества держит контур проверок вокруг неё



Истоки нейрослопа

Весна 2025: Anthropic выпустил Claude Code, и завертелось.

224K

строк - кодовая база разъелась к осени

105 → 1

пакетов не проходили проверку типов: октябрь 2025 → сейчас

Изрядная часть - дубли, брошенные ветки (не гита - кода), забытые эксперименты.

Сейчас все коммиты собираются. Но это сейчас.

Машина ошибается иначе

go vet, staticcheck, golangci-lint подключил почти сразу. Сработали наполовину: они написаны против человеческих ошибок.

Человек

- помнит, что он делал 15 минут назад
- его ошибки индустрия десятилетиями ловит линтерами

Модель

- ошибки растут из короткого контекста
- бесконечные дубли одного и того же
- галлюцинации
- ненужные неиспользуемые ветки

Маскировка непонимания

Самый опасный для финтеха класс.



Группа правил под этот класс: error-masking, fallback-return, log-and-return-zero, empty-struct-return, constructor-swallows-nil-dep. Ни одна не нужна против человека.

50 велосипедов

Схема простая: заметил класс проблем - пишу анализатор.



Универсальные правила - в glint (открыт, MIT), продуктовая специфика - в проектный линтер.

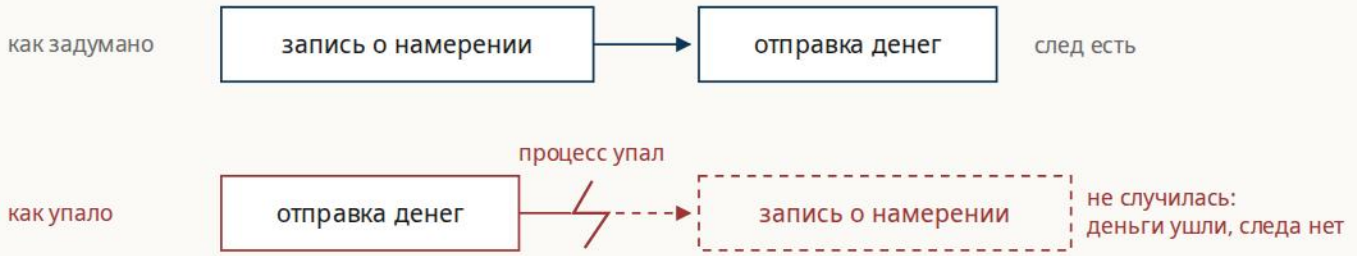
Как баг стал правилом



Контур замкнут: модель находит баги, пишет правило, сама читает выхлоп и дотачивает. Моего участия - на пару решений.

Правило, которое не положить в staticcheck

Платёжный проект: команда провайдеру не должна уходить раньше, чем в своём хранилище появилась запись о намерении её отправить.



- Оба вызова по отдельности корректны, типы сходятся, ошибки обработаны
- Дефектом является их порядок
- Чтобы это поймать, надо знать, как в этом проекте называются провайдеры и хранилища

Методика замера

один и тот же прибор на все точки - заведомо строже того, что существовало тогда



"Тяжёлые" - потенциальная потеря денег, сломанная логика, падение в рантайме. Через всю историю проекта, без единого исключения.

Кривая ступенчатая

Каждый обрыв - событие: январская замена анализаторов и чистка, апрельский разбор god-object-ов, июльские доменные правила, августовское снятие исключений.

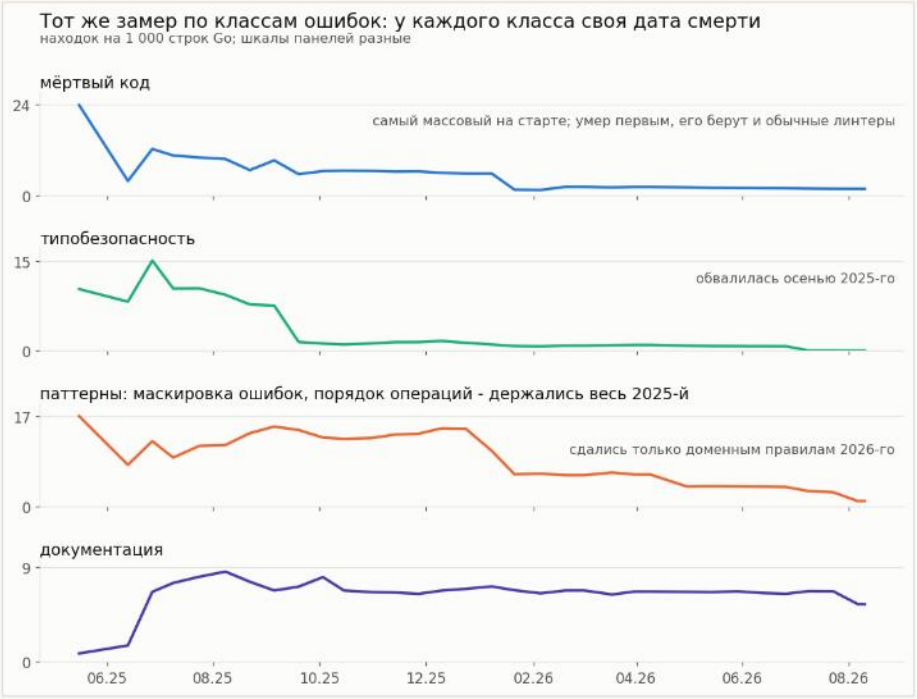
Качество не улучшается от того, что модели становятся умнее. Оно улучшается, когда контур проверок дотягивается до новых мест.



У каждого класса своя дата смерти

- Мёртвый код - самый массовый на старте, умер первым: его берут даже обычные линтеры
- Типобезопасность обвалилась осенью 2025-го
- Маскировка ошибок продержалась весь 2025 год и сдалась только доменным правилам
- Документация единственная светится до сих пор

Ни один класс не умер сам: у каждого своя дата смерти и свой убийца.



Заповедник багов

Замер выживаемости: почти половина строк не менялась с 2025 года. Я был уверен - хлам, руки не дошли.

Оказалось: выживший код лежал ровно в каталогах, которые я сам исключил из проверок. Внутри сохранилось всё, что контур давно вычистил из остального проекта.

Жемчужина коллекции - 6 тестов вида:

```
require.True(t, true, "архитектурные принципы должны соблюдаться")
```

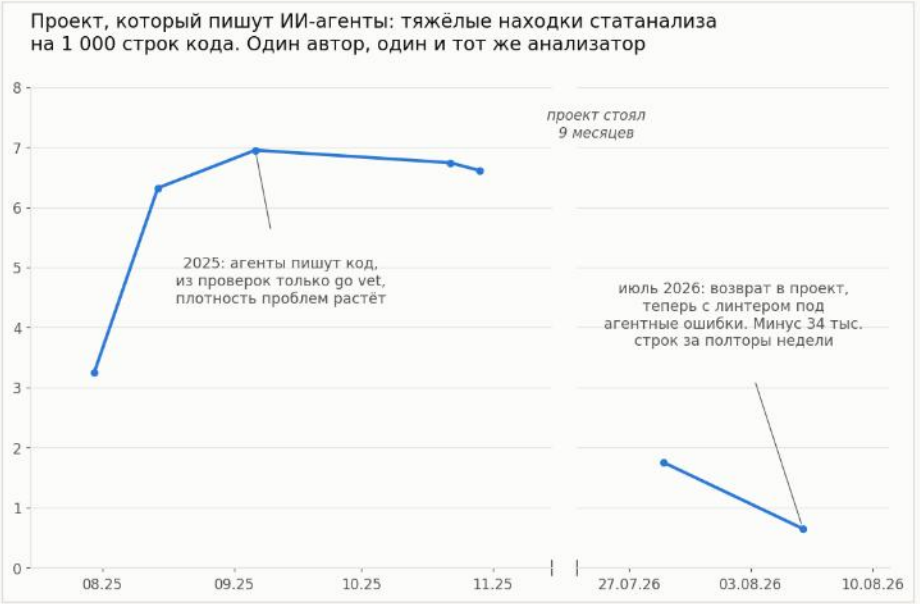
Зелёные всегда, числятся покрытием, не проверяют ничего.

В августе снял старые исключения: две недели разгребал, **0.70 → 0.38**

Контрольная группа

Третий проект: те же месяцы 2025-го, те же модели, тот же я. Одно отличие - из проверок только go vet.

- Плотность тяжёлых находок росла: **3.25 летом → 6.95 к осени**
- Проект стоял 9 месяцев
- Вернулся с линтером: 12 волн чистки за полторы недели, минус 34К строк, на выходе 0.65



Красная черта

А как хорошо я писал без агентов? Крипто-бот 2024 года, 31К строк Go: он торговал моими собственными деньгами, цена ошибки - весь капитал.

- 2.32 тяжёлых находки на тысячу строк
- 21 небезопасный type-assertion, 12 замаскированных ошибок
- 8 раз деньги во float в JSON-контракте

Агентный код сейчас по этой метрике чище **в 6 раз**.



Кто первым узнаёт о баге

Git-история не видит того, что контур поймал до коммита. Это пятно измерено по логам агентных сессий:

132

прогона линтера с находками за май - большой проект

203

эпизода за две недели: фича → находка → правка → зелёное → коммит

Метрика зрелости для любой команды: кто первым узнаёт о дефекте - проверка или клиент?

Где линтер не спасает

478 → 18

находок magic-number → полезных: ранние правила шумели безбожно

6.4%

исторических багов поймал бы линтер: выборка 566 fix-коммитов, часть совпадений случайна

Свежий пропущенный баг: деньги списались, на паре страниц сумма как до вывода - три слоя защиты промолчали разом.

Идеальных инструментов нет, работа над качеством не заканчивается.

Выводы

Качество перестало быть свойством того, кто пишет код, и стало свойством контура, в котором код пишется.

- Агенту не нужно быть безошибочным: ошибки не доживают до коммита
- Контуру безразличен автор: код живого разработчика прошёл те же проверки
- Спор "пишет ли ИИ лучше человека" теряет смысл



Повторить у себя

Инструмент открыт:
github.com/aiseeq/glint

- Режим прогона по некомпилируемой истории
- Скрипт замера в tools/history
- Кривая по своему репозиторию строится за вечер

Статья со всеми цифрами:
habr.com/ru/articles/1068920

Вопросы?



github.com/aiseeq/glint